IUT d'Aix-Marseille Université – site d'Arles
Département Informatique
2ème année

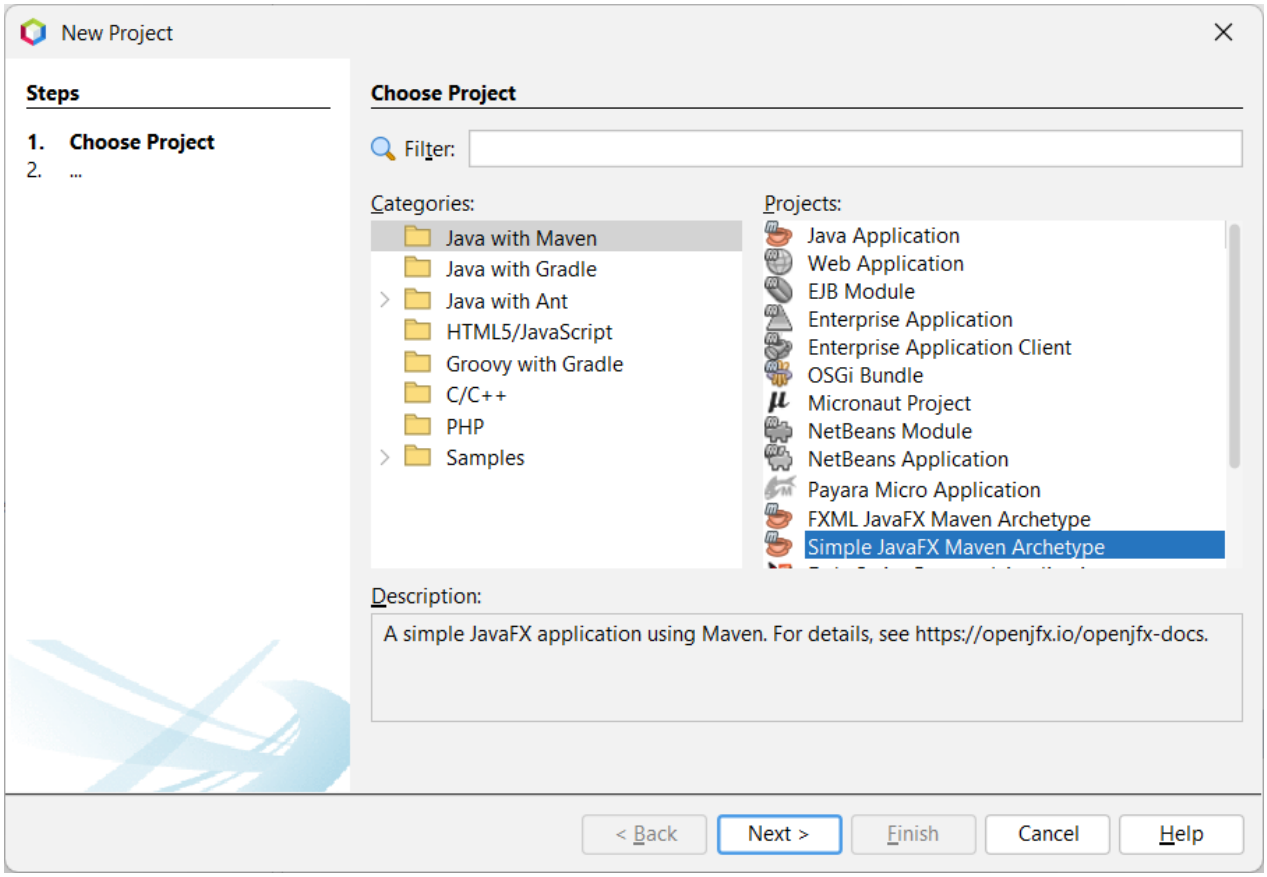
R3.04 – Qualité de développement

TP n°5

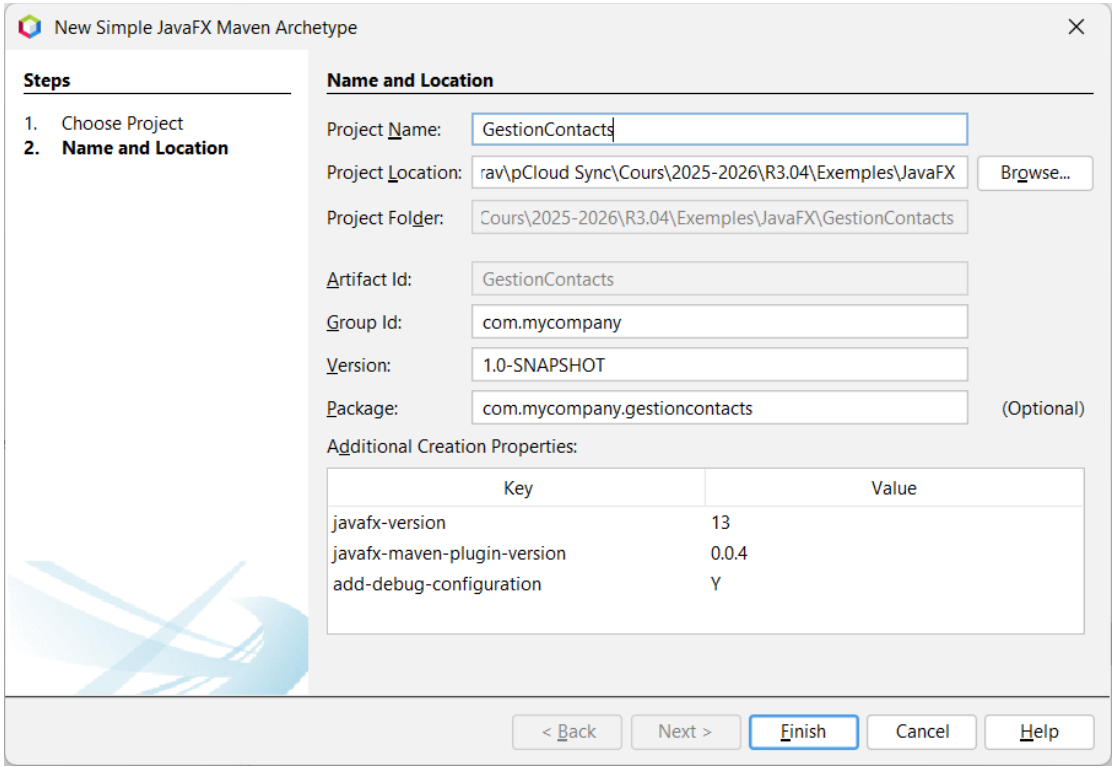
Gestionnaire de contacts avec JavaFX

Vous allez utiliser **JavaFX** pour créer une application de gestion de contacts dotée d'une interface graphique. Vous n'utiliserez pas FXML et SceneBuilder, vous allez dans ce TP coder toute l'UI en Java.

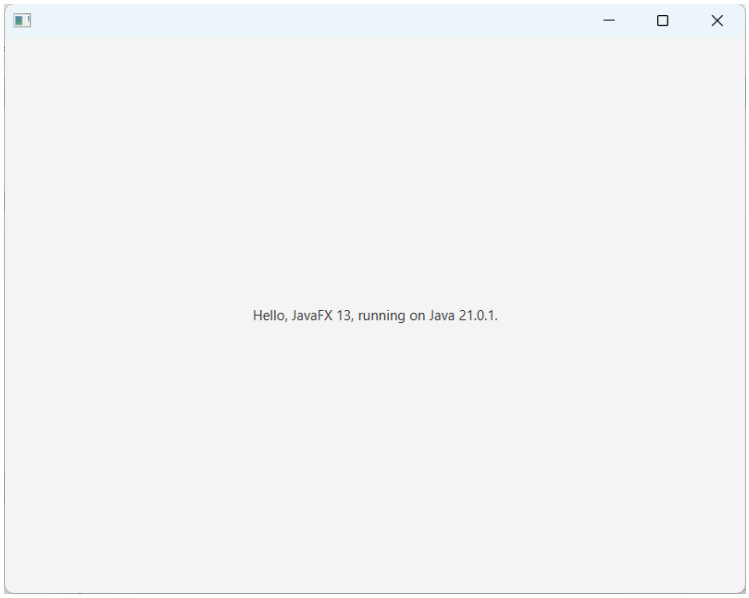
1. Dans Netbeans, créez un projet et dans la catégorie « Java with Maven » choisissez « Simple JavaFX Maven Archetype » (et pas « FXML JavaFX Maven Archetype »).



Appelez votre application GestionContacts :



2. L'assistant vous crée une base de code fonctionnelle, avec une classe **App** qui hérite de la classe JavaFX **Application**, désignant le point d'entrée de votre application. Dans cette classe, un objet **stage** de classe **Stage** est créé, correspondant à la fenêtre principale de votre application. Il contient un objet **scene** de classe **Scene** (= l'ensemble des widgets de la fenêtre) ne contenant pour le moment qu'un label :



3. La fenêtre n'a pour le moment pas de titre. Dans la méthode **start()**, ajoutez un titre à la fenêtre avec la méthode **setTitle()** de la classe **Stage** :

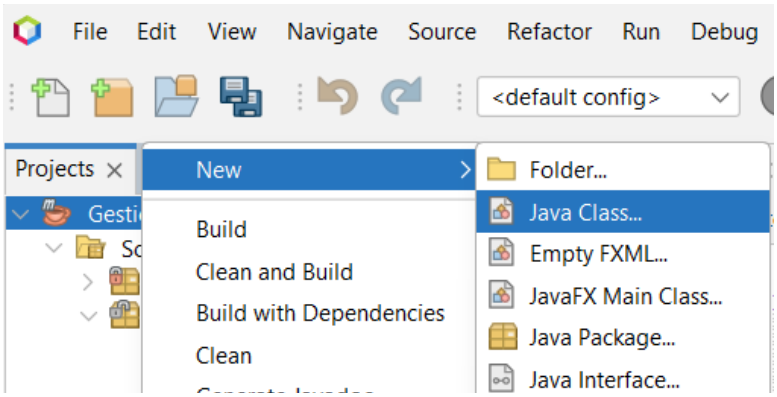
```
stage.setTitle("Gestion de contacts");
```

4. Dans la méthode **start()**, enlevez le label et son ajout à la **scene**. Créez un objet de classe **BorderPane** vide pour le moment (voir chapitre 11 diapo 39) et ajoutez-le à la scène à la place du **StackPane**.

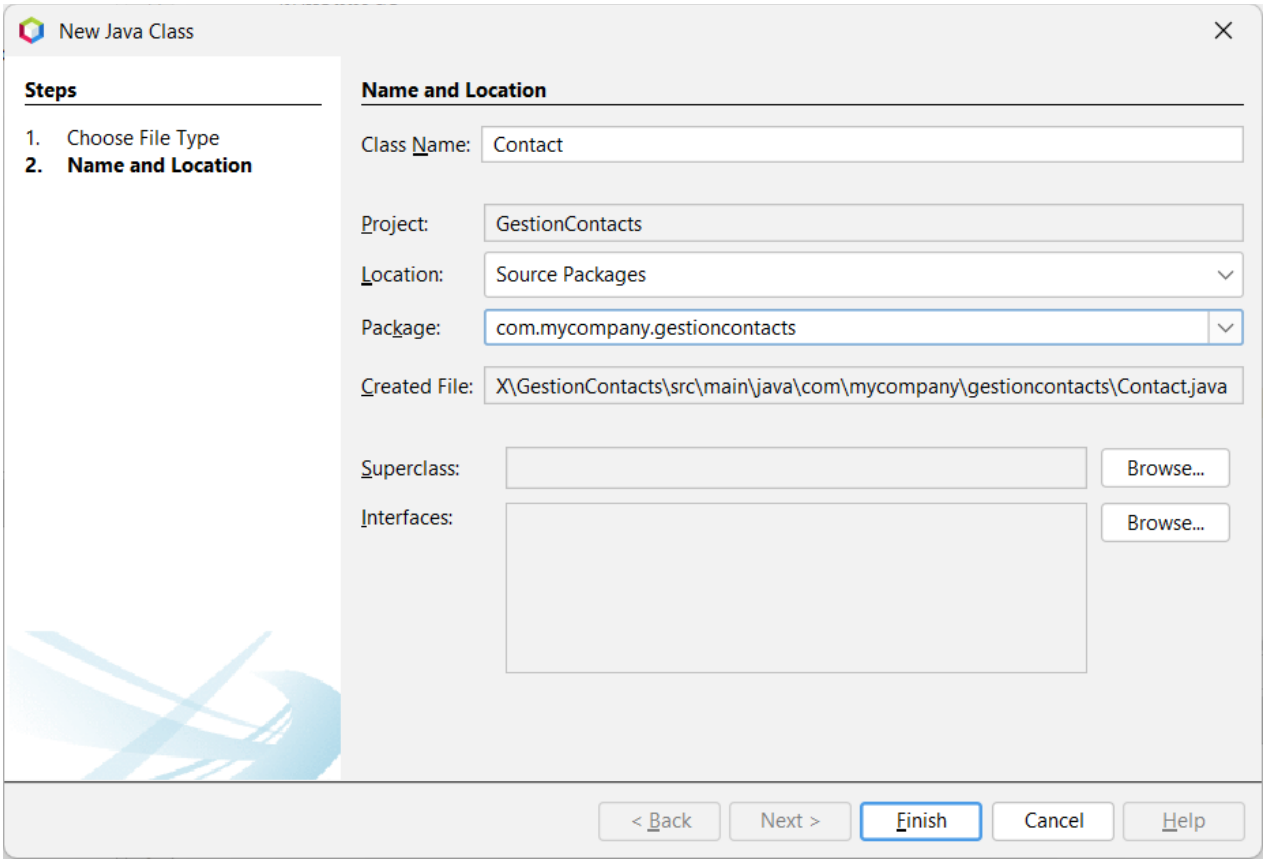
```
import javafx.scene.layout.BorderPane;

BorderPane borderPane = new BorderPane();
var scene = new Scene(borderPane, 640, 480);
```

5. Faites un clic droit sur le nom du projet et sélectionnez **New → Java Class**.



Appelez votre classe **Contact** et changez le nom de son package pour qu'il corresponde à celui de votre fichier **App.java**, dans mon exemple **com.mycompany.gestioncontacts** :



6. Ajoutez à cette classe **Contact** les données membres **lastName, firstName, email, phone**. Plutôt que d'utiliser le type **String**, vous utiliserez pour ces 4 données membres le type **StringProperty** afin que ces données puissent être observables (voir chapitre 11 diapo 54-61) :

```
import javafx.beans.property.SimpleStringProperty;
import javafx.beans.property.StringProperty;

private StringProperty lastName  = new SimpleStringProperty("");
```

Le constructeur de **Contact** recevra en paramètre des **String**, vous les convertirez en **StringProperty** dans le constructeur avec la méthode **set()** :

```
public Contact(String ln, String fn, String em, String ph) {
    lastName.set(ln);
    …
```

Fournissez des getters et setters pour chaque champ. Par exemple pour **lastName** :

```
public String getLastName() { return lastName.get(); }
public void setLastName(String v) { lastName.set(v); }
public StringProperty lastNameProperty() { return lastName; }
```

Attention à la syntaxe de ce dernier getter **lastNameProperty**, il faut qu'il s'appelle exactement comme ça (nom de la variable + Property). En effet, à la question 8 vous allez déclarer une table où vous associerez les colonnes aux champs de la classe **Contact** comme ceci :

```
TableColumn<Contact, String> lastNameCol = new TableColumn<>("Nom");
lastNameCol.setCellValueFactory(new PropertyValueFactory<>("lastName"));
```

Ici, PropertyValueFactory<>("lastName") résout le nom du getter de StringProperty à utiliser pour obtenir la donnée en utilisant la **réflexion** (la capacité d'un programme à inspecter et manipuler sa propre structure au runtime), en cherchant un getter qui s'appelle **lastName** + **Property**, c'est-à-dire ici **lastNameProperty**.

7. Dans la classe **App**, déclarez une liste observable de **Contact** :

```
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;

ObservableList<Contact> contacts = FXCollections.observableArrayList();
```

Le fait que ce soit une **ObservableList** et pas une simple **List** est très important pour la suite. Nous allons mettre cette liste dans une **TableView** à la question suivante, et cette liste jouera le rôle de table de vérité : toute modification dans cette liste (ajout, modification, suppression) sera répercutée dans l'affichage de la **TableView**.

8. Créez une **TableView<Contact>** avec 4 **TableColumn<Contact,String>** (Nom, Prénom, Email, Téléphone) (voir chapitre 11, diapo 32). Par exemple pour la première colonne :

```
TableColumn<Contact, String> firstNameCol = new TableColumn<>("Prénom");
firstNameCol.setCellValueFactory(new PropertyValueFactory<>("firstName"));
```

Vous aurez besoin d'importer ces classes :

```
import javafx.scene.control.TableView;
import javafx.scene.control.TableColumn;
import javafx.scene.control.cell.PropertyValueFactory;
```

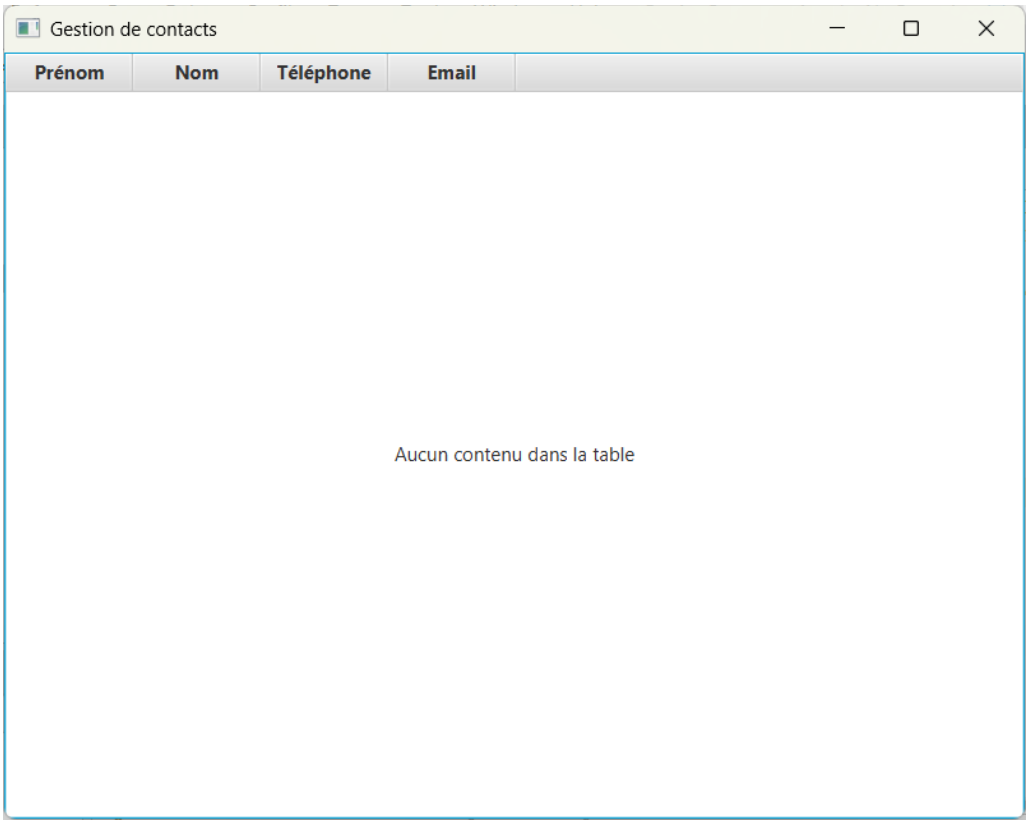
Ajoutez les colonnes à la table :

```
table.getColumns().addAll(firstNameCol, lastNameCol, phoneCol, mailCol);
table.setItems(contacts);
```

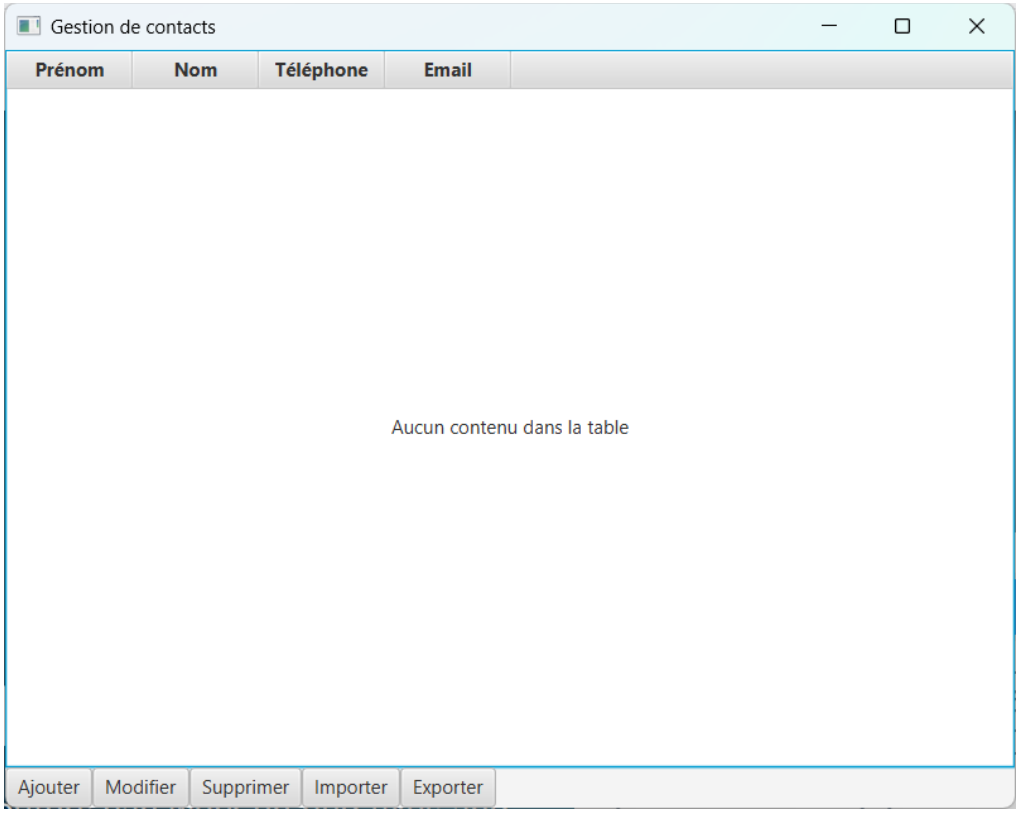
Ajoutez la table au centre de la **borderPane** :

```
borderPane.setCenter(table);
```

La table devrait apparaitre ainsi, vide pour le moment :



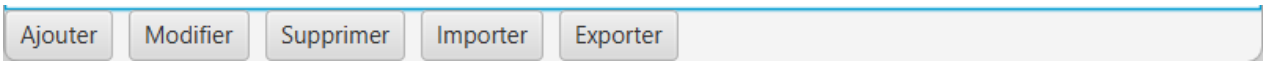
9. Vous allez ajouter une barre d'action en bas de la fenêtre, qui contiendra des boutons permettant de faire des actions sur le contenu de la table. Créez un objet de type **HBox** (voir chapitre 11 page 36) et placez-le en bas de la fenêtre avec borderPane.setBottom(). Créez 5 boutons « Ajouter », « Modifier », « Supprimer », « Importer », « Exporter » et placez-les dans ce **HBox**.



10. Vous pouvez constater que les boutons sont collés entre eux dans la **HBox**. On peut les séparer par un espace en fournissant sa valeur au constructeur de **HBox**. Essayez par exemple :

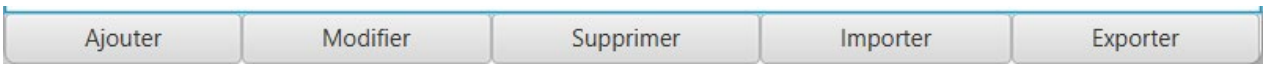
```
HBox hbox= new HBox(8);
```

Ce qui séparera les boutons par 8 pixels :



On peut aussi faire en sorte que les boutons occupent toute la place disponible dans le **HBox** en faisant pour chacun d'entre eux, par exemple pour le bouton **buttonAdd** :

```
buttonAdd.setMaxWidth(Double.MAX_VALUE);  // autorise l'agrandissement
HBox.setHgrow(buttonAdd, Priority.ALWAYS);    // répartit l'espace dans HBox
```



11. Ajoutez à la classe **App** une méthode qui ouvrira une nouvelle fenêtre dans laquelle l'utilisateur pourra entrer ou modifier les informations d'un contact :

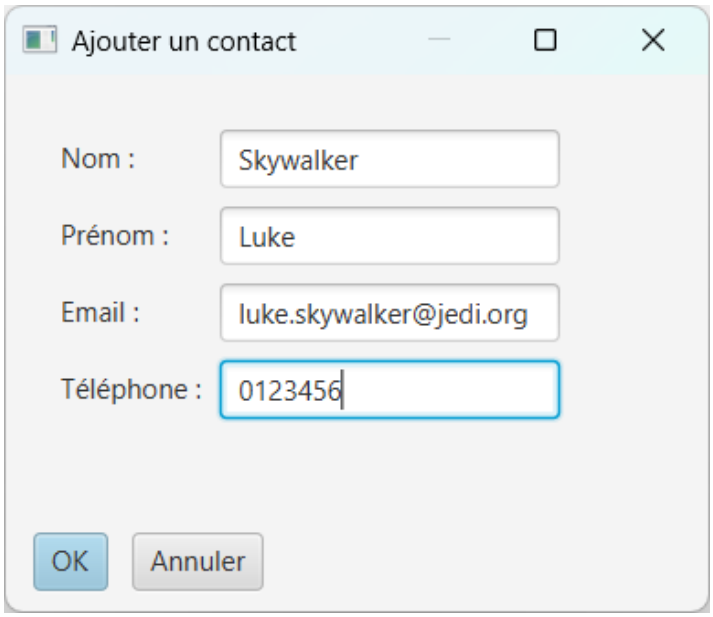
```
openEditor(Contact toEdit)
```

Dans cette méthode, créez une nouvelle fenêtre de classe **Stage**.

```
Stage dlg = new Stage();
```

Ajoutez 4 **TextField** et leurs **Label** pour permettre à l'utilisateur d'entrer un nom, prénom, numéro de téléphone et adresse mail. Si l'objet **toEdit** reçu en paramètre par la méthode **openEditor()** n'est pas **null**, alors initialiser les **TextField** avec les champs de cet objet via leur méthode **setText()**.

Ajoutez des **Button** OK et Annuler dans une **HBox**.



Pour gérer le bouton Annuler, utilisez sa méthode **setOnAction()** pour fermer la fenêtre :

```
cancelBtn.setOnAction(e -> dlg.close());
```

Pour gérer le bouton Ok, utilisez sa méthode **setOnAction()** dans laquelle vous récupérez les valeurs entrées dans les **Textfield** avec leur méthode **getText()**, par exemple :

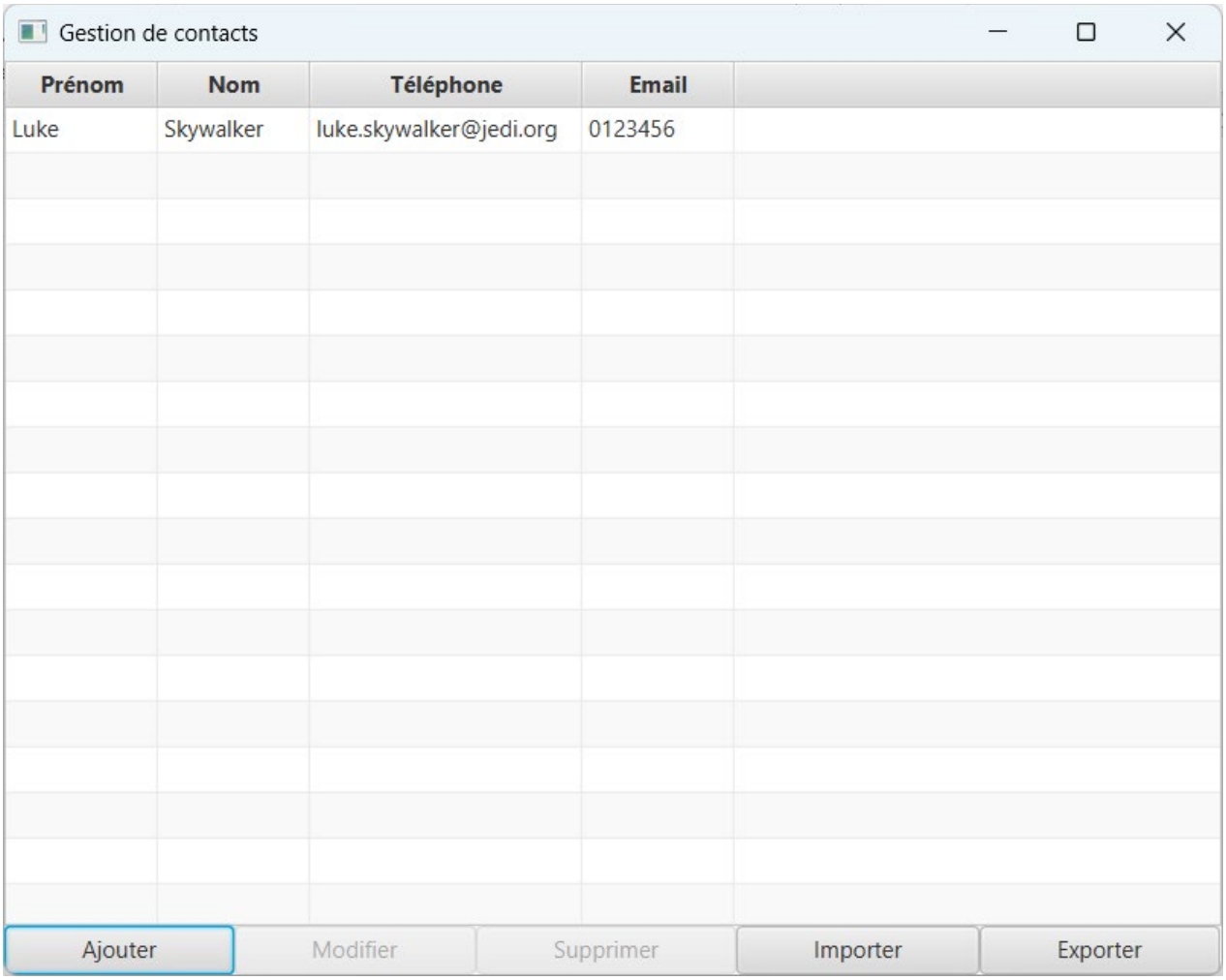
```
String ln = textfieldLastName.getText()
```

Deux possibilités :

- Si **toEdit** vaut **null**, c'est qu'on a appelé **openEditor()** pour la création d'un nouveau contact. Ajoutez donc un nouveau **Contact** dans la liste **contacts** : **contacts.add(new Contact(ln, fn, em, ph));**

- Si **toEdit** ne vaut pas **null**, c'est qu'on a appelé **openEditor()** pour la modification du contact existant **toEdit**. Mettez les 4 valeurs des **Textfield** dans les champs de l'objet **toEdit**.

On voit tout l'intérêt d'avoir utilisé **ObservableList** comme type pour la liste **contacts** : en ayant ajouté un nouvel élément dans cette liste, il apparaît automatiquement dans la **TableView** :



12. Pour appeler la méthode **openEditor()** lorsqu'on clique sur le bouton Ajouter, écrivez dans la méthode **Start()** :

```
buttonAdd.setOnAction(e -> openEditor(null));
```

13. Pour appeler la méthode **openEditor()** lorsqu'on clique sur le bouton Modifier, écrivez dans la méthode **start()** :

```
buttonModify.setOnAction(e -> {
        Contact c = table.getSelectionModel().getSelectedItem();
        openEditor(c);
});
```

Pour faire en sorte que ce bouton soit grisé lorsqu'aucune ligne n'est sélectionnée dans la table, ajoutez dans **start()** :

```
buttonModify.disableProperty().bind(
        table.getSelectionModel().selectedItemProperty().isNull()
);
```

14. Bouton **Supprimer** : faites en sorte qu'il soit grisé lorsqu'aucune ligne n'est sélectionnée dans la table. Pour supprimer un élément sélectionné :

```
Contact c = table.getSelectionModel().getSelectedItem();
contacts.remove(c);
```

15. Bouton **Exporter** : faites apparaître un sélecteur de fichier filtrant les fichiers d'extension .csv (voir chapitre 11 diapo 78). Si l'utilisateur valide, créer un fichier texte au nom choisi, sauvez une première ligne d'en-tête avec « nom ; prénom ; email ; téléphone », puis sauvez une ligne par **Contact** en séparant ses données membres par des points virgules.

16. Bouton **Importer** : faites apparaître un sélecteur de fichier filtrant les fichiers d'extension .csv. Si l'utilisateur valide, ouvrez le fichier texte au nom choisi. Lisez la première ligne, si elle ne vaut pas « nom ; prénom ; email ; téléphone », alors affichez un message d'erreur (voir chapitre 11 diapo 73) et quittez la fonction. Sinon, lisez chaque ligne et créez pour chacune d'entre elle un objet **Contact** que vous ajouterez dans la liste **contacts**.